Claude Code 最佳实践合集

2026年4月 | 92篇实践内容 上手必备

```
// best_practices.config.js
module.exports = {
  files: 92,
  analysis: 'deep'
};
```

内容数

92

- Continue cc-studio pr... Dec 30
- Work on dithered project Dec 11
- Request Assistance for... Dec 10

一起 Vibe

小红书号：LetsVibeCoding

扫描二维码，在小红书找到我

目录

第 0 部分：如何使用这本手册

第 1 部分：安装与入门

Day 0：第一天把 Claude Code 跑起来

macOS 安装

Linux 安装

Windows 安装

第 2 部分：核心概念与最佳实践

Subagents 最佳实践

Skills 最佳实践

Commands 最佳实践

Memory 最佳实践

Settings 最佳实践

MCP 最佳实践

CLI 启动参数最佳实践

第 3 部分：关键解释与边界

Agents / Commands / Skills 边界解释

全局 vs 项目级设置

Monorepo 里的 Skills

Agent Memory 解释

使用量、速率限制与 Extra Usage

高级工具使用模式

浏览器自动化 MCP 选择

Agent SDK vs CLI：系统提示与一致性

LLM 日常退化：神话 vs 现实

第 4 部分：实现与样例

最小工作流样例：Command → Agent → Skill

Commands 实现样例

Skills 实现样例

Subagents 实现样例

Agent Teams 实现样例

Scheduled Tasks 实现样例

第 5 部分：工作流与协作

Cross-model workflow

RPI workflow

Agent Teams 提示词样例

第 6 部分：社区经验与导读

Claude Code 团队的 10 条实战建议

Boris 的 12 条 Claude Code 定制建议

Boris 的 13 条 Claude Code 日常工作流建议

Boris 的 15 个隐藏但高价值的 Claude Code 功能

Code Review 与 Test Time Compute：Boris 的两条高信号建议

为什么要 Squash Merge，以及为什么 PR 要更小

Thariq：Anthropic 内部是怎么用 Skills 的

Boris 上 Lenny's Podcast：编码问题被解决之后，会发生什么？

Boris 上 The Pragmatic Engineer：Claude Code 是怎么做出来的

Boris 上 Ryan Peterman：职业成长、产品判断与 Claude Code

Boris 上 YC：为什么要为 6 个月后的模型设计

Cat 与 Boris 上 Every：Claude Code 是如何被工程化出来的

第 7 部分：哪些值得直接抄，哪些不要直接抄

第 8 部分：7 天学习路径与附录

第 0 部分：如何使用这本手册

这本手册的目标，不是带你看懂这个仓库本身，而是带你学会 Claude Code 这套工作方式。

如果你是第一次接触 Claude Code，建议从第 1 部分和第 2 部分开始。

如果你已经在用 Claude Code，但想把它真正接进项目和团队协作，重点读第 3 到第 6 部分。

第 1 部分：安装与入门

这一部分只解决一个问题：先把 Claude Code 真的跑起来。

先别急着折腾配置、agent 或 workflow；把安装、验证和第一次登录走通，后面所有内容才有意义。

这部分内容按“先总览，再分平台”的顺序组织，适合作为你第一次上手时的安装讲义。

Day 0：第一天把 Claude Code 跑起来

这篇只做一件事：

> 带你把 Claude Code 安装好、验证好、能真正开始用。

第 1 步：按系统安装

请选择你的系统：

系统	指南
Windows	Windows 安装
Linux	Linux 安装
macOS	macOS 安装

装完后继续下一步。

第 2 步：验证安装

```
node --version
claude --version
```

你希望看到的是：

- `node --version` 至少是 `v18.x`
- `claude --version` 能正常返回版本号

第 3 步：第一次登录

第一次运行：

```
claude
```

Claude Code 会让你选择登录方式。

方式 1：订阅账号

- 选择 Claude.ai account
- 浏览器会打开授权页
- 登录并授权
- 回到终端即可

方式 2：API Key

如果你的团队管理员已经邀请你，或者你自己有 key：

- 选择 Anthropic API Key
- 粘贴 `sk-ant-` 开头的 key
- 之后 Claude Code 会记住它

新手提醒

- 第一天先别急着折腾 `.claude/`
- 先把工具跑起来
- 先感受一轮默认体验
- 再继续读更系统的内容

macOS 安装

第 1 步：打开 Terminal

- 按 `Cmd + Space`
- 输入 `Terminal`
- 回车

第 2 步：确认 Homebrew

```
brew --version
```

如果提示 `command not found`，先安装 Homebrew：

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

第 3 步：安装 Claude Code

```
brew install --cask claude-code
```

第 4 步：验证

```
claude --version
```

装好后继续做登录和后续学习。

Linux 安装

你需要什么

- `Node.js v18+`

- `npm`

推荐装法：走 Node 官方页面

最省心的方式是打开：

- <https://nodejs.org/en/download>

然后选择：

- `Linux`
- `fnm`
- `npm`
- 默认 LTS 版本

页面给出的命令通常类似这样：

```
curl -fsSL https://fnm.vercel.app/install | bash
source ~/.bashrc   # 或 source ~/.zshrc
fnm install 24
```

> 具体版本号以网页显示为准。

装完后验证 Node

```
node --version
npm --version
```

安装 Claude Code

```
npm install -g @anthropic-ai/claude-code
```

验证 Claude Code

```
claude --version
```

常见问题

- 如果 `claude` 找不到，先检查 npm 全局 bin 是否在 `PATH` 里
- 如果用系统包管理器装的 Node 太旧，换回官方 LTS 路线
- 在 WSL 里也可以按这份流程走

Windows 安装

第 1 步：安装 Node.js

1. 打开 <https://nodejs.org>
2. 点击 Download Node.js (LTS)
3. 下载 `.msi`
4. 双击运行并按默认选项安装

第 2 步：验证 Node.js

打开新的 PowerShell 或 Windows Terminal：

```
node --version
npm --version
```

第 3 步：安装 Claude Code

```
npm install -g @anthropic-ai/claude-code
```

如果遇到权限问题，用管理员权限打开终端再试。

第 4 步：验证

```
claude --version
```

装好后继续做登录和后续学习。

第 2 部分：核心概念与最佳实践

这一部分负责建立 Claude Code 的基本脑图。

如果 `command / agent / skill / memory / settings / MCP / hooks` 这些词你还没有清楚分层，先把这部分吃透。

Subagents 最佳实践

先讲人话

subagent 适合这样的任务：

- 这件事不是一句话能做完，而是要自己查、自己判断、自己跑几步。
- 你不想让中间过程把主对话上下文塞爆。
- 你希望它带着专门权限、专门记忆、专门 skill 去工作。

如果只是一个简单 prompt 模板，别急着上 subagent。
subagent 用得好是“隔离复杂度”，用不好就是“把每件小事都搞成多进程”。

什么时候该用

- 做多步研究
- 做带权限隔离的任务
- 把重度上下文任务拆出去
- 给某类任务一个长期稳定的人设和边界
- 让 agent 预加载特定 skill

典型例子：

- 代码审查 agent
- 文档整理 agent
- presentation 专用 agent
- 只负责取数的 weather / time agent

什么时候不该用

- 只是想把一句 prompt 存起来
- 只是想让用户手动触发一串固定步骤
- 这个任务更像 command 或 skill
- 任务简单到主线程直接做更快

你真正要理解的 5 个字段

字段	它决定什么	中文理解
`description`	Claude 什么时候会想到调用它	不是功能介绍，而是触发条件
`tools`	它手里能用哪些工具	agent 的能力边界
`permissionMode`	它遇到操作时有多大自主权	安全边界
`skills`	它启动时脑子里预装了什么	预加载知识
`memory`	它会把哪些经验留下来	长期记忆边界

常见误区

误区 1：subagent 越多越高级

不是。
好仓库的特征不是 agent 多，而是 agent 的边界清楚。

误区 2：description 写成“这是一个很厉害的 agent”

没用。
真正有效的写法应该告诉 Claude：

- 什么时候该调用
- 什么时候不该调用
- 这类任务它比主线程更合适在哪里

误区 3：什么都塞进一个 general agent

这样最容易把 agent 做成“第二个主线程”，结果上下文更乱、权限更大、行为更飘。

这个仓库里的代表性做法

Weather Agent

这个仓库用 `weather-agent` 做了一个很经典的教学样例：

- command 负责与用户交互
- agent 负责自治取数
- skill 提供可复用知识

看这里：

- implementation/claude-subagents-implementation.md

- orchestration-workflow/orchestration-workflow.md

Presentation Curator

这个仓库还有一个很有代表性的设计：

- 对 `presentation/**` 目录的改动，不让主线程直接写
- 而是通过 rule 强制交给专门 agent

这比在 `CLAUDE.md` 里喊“请小心修改 presentation”要可靠得多。

面向中文用户的实战建议

如果你打算在自己项目里抄一套 subagent 体系，第一版建议只建 2 到 4 个：

1. `research-agent`
2. `review-agent`
3. `docs-agent`
4. 一个与你业务强相关的专用 agent

先把边界做清楚，再考虑扩张数量。

一句话判断

如果任务的核心矛盾是“复杂、多步、需要隔离”，优先想 subagent。
如果核心矛盾只是“重复执行”，优先想 command 或 skill。

Skills 最佳实践

先讲人话

skill 是 Claude Code 里最容易被低估、也最容易被用错的东西。

最简单的理解：

- command 更像“用户手动点火”的流程入口
- agent 更像“丢出去独立干活”的子线程
- skill 更像“Claude 随身带着的一段知识或一套可复用动作”

它不一定很重，但可以非常高频。

skill 最强的地方

- Claude 可以按语义自动触发
- 你可以手动 `/skill-name`
- agent 可以预加载它
- 它可以带 supporting files
- 必要时可以 `context: fork`，把 skill 运行在子 agent 里

这几个特性叠在一起，skill 就从“提示词模板”升级成了“可复用能力单元”。

什么时候该用

- 你经常要重复解释同一类约束或流程
- 你想让 Claude 在合适的时候自动想到某件事
- 你有大段参考资料，不想每次都塞进主 prompt
- 你要给某个 agent 预装一套领域知识

这个仓库最值得抄的 skill 组织法

这个仓库的 weather 相关 skill 用的是很漂亮的一种结构：

```
my-skill/
├── SKILL.md
├── reference.md
└── examples.md
```

为什么这套结构好：

- `SKILL.md` 保持短、聚焦、可读
- 细节放到 `reference.md`
- 预期输入输出放到 `examples.md`

这就是官方文档提到的 progressive disclosure 思路，用中文说就是：

> 主文件只放最有判断价值的东西，重资料按需再看。

两类 skill，要分清

1. 作为“知识注入”的 skill

这类 skill 更像：

- 规范
- 风格
- 领域知识
- 约束集合

Claude 自动触发它时，重点是“知道什么”。

2. 作为“动作单元”的 skill

这类 skill 更像：

- 部署步骤
- 生成某类产物
- 运行固定检查
- 整理某种输出

重点是“要怎么做”。

这个仓库里的 `weather-fetcher` 更偏知识注入，`weather-svg-creator` 更偏动作单元。

常见误区

误区 1：skill 就是短 prompt

不对。
真正高质量的 skill，应该明确：

- 什么时候触发
- 做到什么算完成
- 哪些资料要按需加载
- 哪些行为不要做

误区 2：把所有细节都塞进 SKILL.md

结果就是：

- skill 自己越来越重
- Claude 每次都吃一大段上下文
- 真正重要的判断信息反而被淹掉

误区 3：description 写成标题，而不是触发器

description 最重要的作用不是“ 给人看简介 ”，而是告诉 Claude：

> 什么场景下该调用我

面向中文用户的实战建议

如果你刚开始做 skill，第一批建议从这 3 类做起：

```
explain-code
project-conventions
deliverable-writer
```

先训练 Claude 在你常见的中文协作语境里表现更稳定，再考虑更重的 skill。

一句话判断

如果你想让 Claude 在很多地方都“ 自然想起同一套知识或动作 ”，skill 往往是最划算的抽象。

Commands 最佳实践

先讲人话

command 的核心价值不是“ 更酷的斜杠命令 ”，而是：

> 把你自己一天里会重复做的工作流，变成一个明确、可复用、可分享的入口。

比如：

- `/review-pr`
- `/release-check`

- `/spec-sync`
- `/techdebt`

这些东西如果天天要重复讲一遍，command 就比手打 prompt 更像工程方案。

command 最适合解决什么问题

- 用户明确想手动触发某件事
- 这件事有固定顺序
- 需要调度 skill 或 agent
- 你不想让主对话永远带着这段长说明

command 是“ 入口 ”，不是“ 长期知识载体 ”。

这件事要特别说明

官方这两年的方向，已经越来越把“ 自定义 commands ”和“ skills ”合并到一套能力模型里看。

也就是说：

- `.claude/commands/` 还可用
- 但在产品语义上，command 和 skill 的边界没有过去那么硬

这个仓库仍然保留了三分法：

- commands
- agents
- skills

这是为了教学更清楚。
中文读者在使用时要记住：

> 这套分类法很有帮助，但不是必须按字面理解成 3 个彼此绝缘的世界。

什么时候优先选 command

- 用户应该显式决定是否执行
- 流程有明确起点
- 你需要一个稳定入口调度其他能力
- 你希望这段说明不常驻在主上下文里

什么时候别急着上 command

- 只是想让 Claude 自动记住一条规范
- 只是想存一段经常会自动触发的知识
- 这件事本质上更像 skill

这个仓库里的代表性样例

`/weather-orchestrator` 非常适合做 command 入门样例，因为它刚好体现了 command 的 4 个职责：

1 接收用户请求

2 跟用户补一个必要参数

3 调 agent 去干自治任务

4 再调 skill 生成结果

看这里：

- implementation/claude-commands-implementation.md
- orchestration-workflow/orchestration-workflow.md

中文用户常见误区

误区 1：command 就是把 prompt 存起来

这是最初级的一层。
更好的 command 应该能：

- 组织用户交互
- 限定工具
- 调度 agent / skill
- 约束输出结果

误区 2：每件小事都做一个 command

结果就是 slash 菜单塞满，但没有学习路径，也没有优先级。

command 应该服务“高频工作流”，不是服务“每一个灵感”。

一句话判断

如果你脑子里想到的是“我想给某个工作流做一个明确入口”，那你想的多半就是 command。

Memory 最佳实践

先讲人话

在 Claude Code 里，memory 最常见的误解是：

> “只要有一个巨长的 `CLAUDE.md`，Claude 就应该永远记得所有事。”

现实不是这样。

更实用的理解是：

- `CLAUDE.md` 是项目记忆入口
- `rules` 是可拆分的局部规则
- 子目录的 `CLAUDE.md` 是更细颗粒度的上下文

关键不是“写多长”，而是“放在哪一层”。

root `CLAUDE.md` 该放什么

- 项目级工作方式
- 共享约束
- 哪些目录有特殊规则
- 回答问题时的优先信息源

不建议塞进去的：

- 冗长历史背景
- 一次性任务说明
- 很细的局部页面规范

子目录 `CLAUDE.md` 什么时候有用

当你的仓库是 monorepo，或者不同模块真的有明显不同的工作方式时，它就很有用。

上游最值得抄的一点，是把 memory 的加载方式讲得很清楚：

- 向上找祖先目录时会加载
- 向下的子目录 memory 是按需加载
- 兄弟目录不会无缘无故混进来

这件事对中文用户尤其重要，因为很多人会把所有说明都堆在根目录，最后谁都不爱看。

这个仓库怎么做的

根 `CLAUDE.md` 做了三件事：

1. 解释仓库是什么
2. 规定读这个仓时优先查哪里
3. 说明关键系统，比如 weather workflow、hooks、presentation rule

这就是一个比较好的项目级记忆写法：

- 有方向
- 不啰嗦
- 能约束 Claude 的行为

中文团队实战建议

建议把 memory 拆成三层：

层 1：根 `CLAUDE.md`

放团队共享规则和工作方式。

层 2：`.claude/rules/*.md`

放可复用、可拆分、路径相关的规则。

层 3：子目录 `CLAUDE.md`

放真正模块级的差异，不要乱建。

一句话判断

memory 的重点不是“写更多”，而是“让 Claude 在正确的层读到正确的东西”。

Settings 最佳实践

先说这篇要解决什么

这篇不是逐项翻译上游那份 900 多行的 settings
大全，而是想先帮中文读者建立一个更靠谱的“设置脑图”：

- 哪些是运行时控制面
- 哪些是安全边界
- 哪些是团队共享项
- 哪些只是个性化外观

如果这张脑图没立住，后面你再看字段表，也很容易越看越乱。

先讲人话

很多人以为 Claude Code 不听话，是 prompt 不够狠。

更常见的现实是：

- 该放在 `CLAUDE.md` 的东西塞进 prompt
- 该放在 settings 的东西变成了口头要求
- 该用 hooks 或 rules 管的行为，被期待模型自己“记住”

所以 settings 不是装饰文件，而是运行时控制面。

你可以先把 settings 分成 5 层

1. 权限层

你主要在决定：

- 什么默认允许
- 什么需要 ask
- 什么必须 deny

这一层解决的是“安全边界”，不是“体验优化”。

2. 模型与输出层

你主要在决定：

- 默认模型
- effort
- outputStyle
- fast mode

这一层决定 Claude 在你项目里“表现成什么风格”。

3. 工具与集成层

你主要在决定：

- MCP
- hooks
- status line
- plugins

这一层决定 Claude 会接上哪些外部能力。

4. 项目共享层

你主要在决定：

- 什么配置应该跟着项目走
- 什么东西该进 git
- 团队成员打开项目时应该默认获得什么行为

5. 个性化层

比如：

- spinner 文案
- status line 花样
- 某些个人偏好的 local settings

这层重要，但绝对不该先于前 4 层。

最值得先理解的优先级

从中文团队实际使用角度，建议先把这个顺序记住：

1. 安全
2. 一致性
3. 可协作
4. 自动化
5. 个性化

你先把权限边界、模型策略、团队共享约定想清楚，再去调 spinner verbs，长期效果会好很多。

这个仓库的 `.claude/settings.json` 在展示什么

它明显不是“保守企业默认值”，而是一个 power-user preset。

它展示了这些方向：

- 权限开得比较宽，但高风险命令转 ask
- hooks、status line、spinner、plansDirectory 一起配置

- project MCP 更容易启用
- 输出风格和会话体验也被一起纳入控制

这非常适合教学，因为能让读者快速看到 Claude Code 能管到哪里。
但它不等于“你团队该原样照抄”的默认基线。

哪些设置值得优先抄

A. 权限结构

最值得学的是分层思路，不是具体 allowlist。

你至少要明确：

- 哪类命令永远不该自动放行
- 哪类命令可以 ask
- 哪类工具可以默认允许

B. 模型与输出风格

如果你们团队对“Claude 回答要短还是要解释”没有共识，后面的协作会很痛苦。

C. hooks / MCP 开关

不要只会装能力，还要会决定“哪些项目默认启用、哪些先别动”。

D. 项目级 vs local 级

团队共享的是：

- `.claude/settings.json`

个人覆盖的是：

- `.claude/settings.local.json`

这两个界限不清，仓库会很快变脏。

哪些设置先别急着抄

- 纯风格化的 spinner 文案
- 高个性化的 status line 文案
- 没想清楚就把所有 hooks 都挂上
- 还没理解权限边界就打开过多工具

中文团队的实战落地建议

如果你今天要给团队起一版 settings，建议只先管下面这些：

第一版必须明确

- 权限规则
- 默认模型 / effort
- output style

- MCP 是否启用
- hooks 是否启用

第一版可以先不追求

- 好看的 status line
- 个性化 spinner
- 花哨的外观型设置

一个更实用的检查表

在你提交 settings 之前，问自己：

问题	如果回答不清楚，说明还不该提交
我们为什么允许这些工具？	权限边界没想清楚
这些设置是团队共享还是我个人偏好？	层级混了
如果新同事打开项目，会不会被惊到？	默认值过猛
这些 hooks / MCP 真的是高频需求吗？	自动化过度

一句话总结

好的 settings，不是“把所有功能都配出来”，而是：

> 让 Claude 在这个项目里安全、稳定、可协作地工作。

继续读哪里

- reports/claude-global-vs-project-settings.md
- best-practice/claude-memory.md
- best-practice/claude-mcp.md
- 上游完整 settings 参考：<https://github.com/shanraisshan/claude-code-best-practice/blob/main/best-practice/claude-settings.md>

MCP 最佳实践

先讲人话

MCP 不是“Claude 变得更强”的抽象说法，而是：

> 让 Claude 接上外部工具、文档、浏览器、数据库和 API 的标准方式。

但真正的最佳实践不是“接越多越好”，而是：

> 只接你真的高频会用、而且能明显降低幻觉或人工切换成本的那几类。

这个仓库为什么值得看

因为它对 MCP 的态度很克制。

`.mcp.json` 里只放了少量高频项，而不是把 MCP 当收藏夹：

- Context7
- Playwright
- DeepWiki

这其实就是很成熟的判断：

- 研究层：Context7 / DeepWiki
- 浏览器验证层：Playwright

日常最值得先接的几类 MCP

1. 文档类

代表：Context7、DeepWiki

适合：

- 查库文档
- 查第三方 API
- 查仓库结构和 wiki

2. 浏览器 / 调试类

代表：Playwright、Chrome 相关 MCP

适合：

- 页面验证
- 自动化测试
- console / network 调试

3. 图形 / 说明类

代表：Excalidraw 这类

适合：

- 生成架构图
- 产出流程图

配置怎么理解

最关键的是三件事：

1. 放在哪一层

- 项目级：`.mcp.json`
- 用户级：`~/.claude.json`
- agent 级：agent frontmatter

2. 怎么控制启用

和 `.claude/settings.json` 配合：

- `enableAllProjectMcpServers`
- `enabledMcpjsonServers`
- `disabledMcpjsonServers`

3. 怎么配权限

MCP 工具最终还是要落到权限规则里，例如：

- `mcp__*`
- `mcp__context7__*`
- `mcp__playwright__browser_snapshot`

中文团队最值得抄的原则

- 先接高频 MCP，再接“看起来很酷”的 MCP
- 先解决文档真实性和浏览器可见性，再扩展更多玩法
- 不要让 MCP 数量本身变成上下文噪音

一句话总结

好的 MCP 策略不是“全都接”，而是“把最关键的外部能力稳稳接上”。

CLI 启动参数最佳实践

先讲人话

如果你已经开始把 Claude Code
真正纳入日常工作流，启动参数就不是“冷门小知识”，而是你怎么开工的入口层。

最常见的几类启动参数，其实分别在决定：

- 你要恢复哪个会话
- 你用哪个模型
- 你给 Claude 多大权限
- 它要以什么格式输出
- 它要带着什么 system prompt / agent / MCP / 插件起步

最值得先记住的几类参数

1. 会话管理

适合高频使用的：

- `--continue`
- `--resume`
- `--fork-session`
- `--remote`

- `--teleport`

这类参数决定你是从哪段上下文开始工作，而不是从零开新局。

2. 模型与配置

最重要的：

- `--model`
- `--fallback-model`
- `--betas`

如果你经常切场景，这些参数会比在会话里临时改更可控。

3. 权限与安全

重点是：

- `--permission-mode`
- `--allowedTools`
- `--disallowedTools`
- `--dangerously-skip-permissions`

真正的经验不是“把权限全开”，而是让启动方式就带着清楚的安全边界。

4. 输出与格式

最常用的是：

- `--print`
- `--output-format`
- `--json-schema`
- `--verbose`

如果你把 Claude Code 接到脚本、自动化或其他系统里，这一组尤其重要。

5. Agent / MCP / 目录

这类参数决定的是启动时环境：

- `--agent`
- `--agents`
- `--mcp-config`
- `--plugin-dir`
- `--add-dir`
- `--worktree`

对中文用户最实用的判断

如果你只是日常手动用

先掌握：

- `--continue`
- `--resume`
- `--model`
- `--permission-mode`

如果你开始做自动化或脚本接入

重点看：

- `--print`
- `--output-format`
- `--json-schema`
- `--max-turns`

如果你开始做多目录 / 多代理 / 多工具工作流

重点看：

- `--agent`
- `--mcp-config`
- `--plugin-dir`
- `--add-dir`
- `--worktree`

最容易犯的错

- 只会在会话里改设置，不会在启动层固定上下文
- 把危险权限参数当成效率捷径
- 不区分“交互式使用”和“脚本式使用”

一句话总结

启动参数不是冷门附录，它决定了 Claude Code 一开场就站在哪个工作流姿势里。

第 3 部分：关键解释与边界

这一部分回答的是“为什么要这样分层”和“最容易搞混的边界是什么”。

如果你已经会用一点 Claude Code，但总觉得概念模模糊糊，这部分最有价值。

Agents / Commands / Skills 边界解释

TL;DR

最简单的判断：

- `agent`：丢出去独立干活
- `command`：给用户一个明确入口
- `skill`：让 Claude 带着一套知识或动作随时可用

一张表说清楚

机制	最像什么	什么时候优先选它
Agent	独立上下文里的子线程	复杂、多步、需要隔离或自治
Command	用户显式触发的工作流入口	想把高频流程收成 `/xxx`
Skill	可复用知识或动作单元	想让 Claude 自动想起一套能力

为什么这篇报告重要

因为很多中文用户第一次接触 Claude Code 时，最容易混的就是这三样。

常见症状：

- 明明应该做成 skill，却做成 command
- 明明只是一个入口，却做成大而全 agent
- 明明是项目规范，却做成一次性 prompt

这个仓库给出的一个高质量答案

它不是靠“定义”来区分三者，而是靠 weather 样例来区分：

- `weather-orchestrator` 是 command
- `weather-agent` 是 agent
- `weather-fetcher` / `weather-svg-creator` 是 skill

这比抽象讨论好得多，因为你能看到：

- 谁负责用户交互
- 谁负责自治执行
- 谁负责提供可复用知识或动作

作为中文读者，你最该记住的判断线

如果重点是“入口”

选 command。

如果重点是“自治”

选 agent。

如果重点是“复用”

选 skill。

一个现实提醒

官方现在越来越把 custom command 和 skill 放进一套更统一的能力模型里看。
所以这篇报告里保留三分法，是为了帮你理解，不是为了让你死守概念边界。

别做的事

- 不要为了显得高级，把所有流程都 agent 化
- 不要为了省事，把所有知识都塞进根 `CLAUDE.md`
- 不要把 command 当成唯一工作流工具

最后的判断口诀

> 入口选 command，自治选 agent，复用选 skill。
>
> 如果三者都像，那就回到“这件事最主要的矛盾是什么”。

全局 vs 项目级设置

这篇报告回答一个关键问题

到底哪些东西应该放：

- `~/.claude/`
- 项目里的 `.claude/`
- 根目录的 `CLAUDE.md`
- 本地 `settings.local.json`

如果这一层搞不清，Claude Code 很容易被你自己配乱。

最简单的判断标准

放全局

当它属于：

- 你个人的长期习惯
- 跨项目通用偏好
- 不该进 git 的个人配置

放项目级

当它属于：

- 这个仓库的团队协作规则
- 这个项目共享的 hooks / settings / commands / skills
- 应该被版本控制追踪的配置

这个仓库为什么值得看

因为它不只是把 `.claude/`
当配置目录，而是把“全局和项目级的边界”当成一个必须单独讲清楚的主题。

这对中文团队尤其重要，因为最常见的混乱就是：

- 团队约束放进个人全局目录
- 个人偏好误提交进项目仓
- 想做项目级共享，结果改成了自己机器上的局部配置

你先记住这几条就够了

- `settings.local.json` 更像“我自己的项目内覆盖”
- `settings.json` 更像“团队共享默认值”
- `CLAUDE.md` 更像“项目级工作记忆”
- `~/.claude/` 更像“我的个人底座”

对中文 fork 的意义

这份中文仓也采用同样逻辑：

- 中文说明文档进仓
- 运行层配置保留英文
- 本地 `upstream` remote 是本地工作习惯，不是仓库文件
- 同步状态通过 SYNC_STATUS.md 记录，而不是靠口头约定

一句话判断

凡是“团队应该一起看到、一起维护、一起同步”的，优先放项目级。
凡是“只是你自己想这么用”的，优先放全局或 local。

Monorepo 里的 Skills

先讲结论

在 monorepo 里，skill 最重要的不是“多”，而是“放对地方”。

上游这篇报告最有价值的一点，是把 skill discovery 说清楚了：

- 根目录 skill 负责共享能力
- 子目录 skill 负责局部能力
- Claude 会按你正在操作的目录动态发现它们

为什么这件事重要

因为 monorepo 最怕两种极端：

- 所有 skill 都堆在根目录，最后谁都不想看
- 每个 package 各写各的，没人知道从哪触发

实用建议

根目录 skill

适合放：

- 仓库通用规范
- 通用交付格式
- 共用的 review / explain / release 流程

子目录 skill

适合放：

- 某个 package 的框架约束
- 某个模块特有的工具链说明
- 与该目录强绑定的流程知识

这跟 `CLAUDE.md` 的区别

很多人会把 skill 和 memory 混成一回事。

更实用的区分：

- `CLAUDE.md` 解决“这里的通用工作方式是什么”
- skill 解决“遇到这种任务时，Claude 应该调用哪套知识或动作”

一个偏长期记忆，一个偏可调用能力。

中文团队怎么抄

如果你的仓库是 monorepo，第一版建议：

- 根目录保留少量高价值 skill
- 每个业务大目录只给 1 到 2 个 skill
- 先让触发条件写清楚，再扩数量

一句话判断

monorepo 里的好 skill 体系，不是铺满目录树，而是让 Claude 在正确的位置发现正确的 skill。

Agent Memory 解释

这篇报告回答什么

`memory` frontmatter 到底有什么用？

它的核心价值是：

> 让某个 agent 不再每次都从零开始。

它解决的问题

没有 memory 时，agent 每次调用都像第一次来这个仓库。

有了 memory 之后，它可以跨会话积累：

- 代码风格模式
- 常见问题
- review 经验
- 项目习惯

三种作用域

Scope	更适合什么
`user`	你个人跨项目的长期经验
`project`	这个项目团队应该共享的经验
`local`	只属于你自己的项目内经验

什么时候值得用

- code review agent
- 文档审校 agent
- 某类长期重复的质量检查 agent

什么时候别急着用

- 你只是做一次性任务
- agent 的边界还没稳定
- 你还没想清楚应该记住什么、不该记住什么

它和其他 memory 机制的区别

- `CLAUDE.md`：项目级共享工作记忆
- `rules`：路径级模块化约束
- `agent memory`：某个 agent 自己长期积累的经验

这三者不要混。

中文团队最有价值的用法

如果你有一个 review agent，最值得记住的是：

- 项目常见问题模式
- 反复被指出的约定
- 某些模块的历史坑

一句话总结

agent memory 最适合那些“会重复看同一类问题”的 agent。
如果任务不重复，memory 的收益就不会太大。

使用量、速率限制与 Extra Usage

这篇报告解决什么问题

当你开始高频使用 Claude Code，迟早会遇到这些问题：

- 我还剩多少额度？
- 为什么突然慢了或被限了？
- 限额到了之后怎么继续干活？

你先记住 3 个命令

命令	用来干嘛
`/usage`	看当前计划额度和速率限制
`/extra-usage`	限额到了之后继续用，走按量付费溢出
`/cost`	看当前 session 的 token / 花费情况

`/usage`

最适合在这些时候看：

- 你感觉 Claude 变慢了
- 你担心快触顶了
- 你要规划今天剩余工作量

`/extra-usage`

它的意义不是“多一个按钮”，而是：

> 当订阅限额到了，不要让工作流直接中断。

但前提是你清楚：

- 这是额外计费
- 不是订阅内包含
- 应该有预算意识

`/cost`

如果你是 API key 用户，这个命令尤其重要。

它最适合：

- 算一轮 session 到底花了多少
- 比较不同工作流的成本
- 看某个流程是不是在无意义烧 token

对中文用户最现实的建议

- 日常重度使用时，把 `/usage` 当健康检查命令
- 如果你在团队环境里，先想清楚谁可以开 `/extra-usage`
- 如果你在做复杂流程设计，别只看效果，也要看成本

一句话总结

高频使用 Claude Code 之后，配额管理不是财务问题，而是工作流问题。

高级工具使用模式

这篇报告在讲什么

当你开始给 Claude 接越来越多工具时，真正的问题不再是“能不能调用”，而是：

- token 会不会被工具来回调用吃爆
- 搜索结果会不会把上下文塞满
- 工具 schema 够不够清楚
- 模型能不能稳定选对工具、填对参数

这篇报告讨论的就是这些 API 级能力：

- Programmatic Tool Calling
- Dynamic Filtering
- Tool Search Tool
- Tool Use Examples

先讲结论

如果你的工具越来越多，或者中间结果越来越大，这类能力不是锦上添花，而是成本控制手段。

1. Programmatic Tool Calling（PTC）

它解决什么问题

当一个任务要连续调多个工具时，传统 agent 回环会反复把中间结果塞回上下文，token 消耗很快。

它的核心思路

让“多次工具调用 + 中间处理”在程序里发生，而不是每一步都回到模型上下文里。

什么时候值得用

- 多工具串联
- 中间结果很长
- token 成本已经明显上升
- 想把 agent 循环压得更短

2. Dynamic Filtering

它解决什么问题

网页搜索和抓取很容易把大量无关内容一起带回上下文。

它的核心思路

先做动态筛选，再把真正相关的内容交给模型。

适合场景

- 技术文档检索
- 多来源交叉验证
- 网页研究任务

3. Tool Search Tool

它解决什么问题

工具一多，单是工具定义本身就会吃掉上下文，还会影响模型选工具的准确率。

它的价值

不是“又多一个工具”，而是把工具发现这件事本身也做成可调度能力。

4. Tool Use Examples

它解决什么问题

仅靠 schema，往往不够告诉模型：

- 什么时候该填哪些参数
- 哪些组合最常见
- 哪些格式容易出错

所以工具示例很重要。

对 Claude Code 用户最重要的现实意义

这些能力很多更偏 API / Agent SDK 层，但它们会反过来影响你怎么设计 Claude Code 工作流。

最值得学的不是某个名词，而是这条原则：

> 工具越多，越要把“如何降低上下文噪音”当成一等问题。

实战建议

如果你已经开始做 MCP、custom tools 或复杂 agent：

1. 先解决工具太多导致的上下文膨胀
2. 再解决中间结果太长的问题
3. 最后再补工具示例和参数模式

一句话总结

高级工具使用模式的核心，不是让 Claude
“会更多工具”，而是让它在更多工具存在时依然稳、准、便宜。

浏览器自动化 MCP 选择

先讲结论

如果你想给 Claude 一个稳定、可复用、适合自动化测试的浏览器能力层：

- 首选：Playwright MCP
- 次选：Chrome DevTools MCP
- 场景型补充：Claude in Chrome

为什么

Playwright MCP

优点：

- 自动化测试语义成熟
- 跨页面流程更稳
- 更适合脚本化和重复验证

Chrome DevTools MCP

优点：

- 对浏览器内部状态、console、network 观察更强
- 更像调试工具而不是测试框架

Claude in Chrome

优点：

- 更贴近真实浏览器使用体验
- 对“让 Claude 直接在浏览器里操作”这件事很直观

局限：

- 更偏交互式辅助，不是最稳的自动化测试底座

怎么选

你主要做自动化测试

选 Playwright MCP。

你主要做浏览器调试

选 Chrome DevTools MCP。

你主要做“让 Claude 看见真实浏览器”

再考虑 Claude in Chrome。

对中文用户最有价值的启发

不要把“能控制浏览器”当成单一能力。
浏览器自动化至少分三类：

- 调试
- 测试
- 真实交互观察

不同 MCP 擅长的是不同层。

一句话总结

Playwright 更像稳定测试底座，DevTools 更像深调试利器，Claude in Chrome 更像贴近真实操作体验的辅助层。

Agent SDK vs CLI：系统提示与一致性

先讲结论

同一句输入，经过：

- Claude Agent SDK
- Claude CLI（Claude Code）

不保证得到相同输出。

原因不是“模型偷偷换了”，而是这两条入口默认就携带不同的系统提示、工具语境和运行环境。

这件事为什么重要

很多人会在 CLI 里调出一个很顺手的效果，然后想：

> 我把同一句 prompt 扔进 SDK，不就应该一样吗？

现实通常不是这样。

因为模型最终看到的不是“只有你的那一句话”，而是：

- 你的消息
- 系统提示
- 工具说明
- 环境上下文
- 项目记忆
- preset

这几层只要有差异，输出就会漂。

CLI 和 SDK 的真实差别

Claude CLI / Claude Code

CLI 更像一个“已经组装好的工作台”。

默认会更靠近真实工程语境：

- 工具说明
- 编码准则
- 安全规则
- 项目上下文
- CLAUDE.md / settings / hooks 等附加信息

好处是：

- 开箱就能在真实项目里工作
- 工具和环境上下文更完整

代价是：

- 默认行为更重
- 不容易和 SDK 做 1:1 对齐

Agent SDK

SDK 更像一层可编程底座。

好处是：

- 你能自己控制更多东西
- 更适合自建 agent 系统
- 更适合嵌入你自己的产品和服务

代价是：

- 你要自己把很多“ CLI 默认帮你带上的东西 ”补齐

什么时候最容易误判

误区 1：同模型 + 同用户输入 = 同输出

不是。

系统提示、工具集、上下文来源、采样条件，只要有一个不同，输出都可能明显不同。

误区 2：CLI 的好结果可以直接复制到 SDK

也不是。

CLI 的优势很多时候来自“ 整套工程语境 ”，不是单个 prompt 魔法。

如果你想尽量接近一致，需要对齐什么

至少要看这几项：

维度	要不要对齐	说明
系统提示	必须	最大差异源
工具说明	必须	直接影响模型行为

项目上下文	尽量	CLI 通常更重
运行 preset	尽量	尤其是 Claude Code 风格预设
输出格式约束	需要时	会影响回答形态

什么时候该用 CLI

- 你想直接在真实工程环境里工作
- 你想利用现成交互和项目上下文
- 你更关心“马上可用”而不是“完全自定义”

什么时候该用 SDK

- 你要自己搭 agent 系统
- 你要可编程编排流程
- 你要把 Claude 接进自己的应用、服务、平台里

对中文开发者最实用的启发

别把 CLI 和 SDK 看成“同一件东西的两个壳”。

更准确的说法是：

- CLI 是偏成品的工作台
- SDK 是偏底座的可编排能力

如果你要追求两边“一致”，首先要问：

> 我到底是在复用模型，还是在复用整套工作环境？

一句话总结

CLI 和 SDK 共用的是模型，不是默认工作环境。
真正决定输出是否一致的，是你有没有把“环境层”也一起对齐。

LLM 日常退化：神话 vs 现实

先讲结论

很多人都有这种感觉：

> 昨天的 Claude 很神，今天怎么像降智了？

这个感觉不一定全是错觉，但也不能简单归因成“模型偷偷被削弱了”。

更准确的理解

模型权重通常不会天天变。
但用户感受到的质量波动，确实可能来自很多别的层：

- 基础设施 bug
- 路由差异

- 系统提示变化
- 后训练更新
- 上下文污染
- 采样随机性
- 人的认知偏差

真实波动通常来自哪几层

1. 推理基础设施

包括：

- 路由错误
- 编译器 / 硬件问题
- 部分请求落到了不理想路径

这类问题最麻烦的地方是：

- 用户只能感受到“今天怪怪的”
- 但表面上模型名字没有变

2. 系统提示 / 后训练 / 服务配置

即使模型权重没变，下面这些也可能变：

- 系统提示
- provider 侧包装逻辑
- 后训练小更新
- 默认策略

结果就是：

- 同样的输入，行为边界会有细微变化

3. 上下文与任务本身

很多“模型退化感”，其实是：

- 这次任务更难
- 上下文更脏
- prompt 更模糊
- 你刚好踩中了模型弱点

为什么“全是心理作用”也不对

因为真实世界里，基础设施和配置层的问题确实会造成波动。

所以这不是二选一：

- 不是“全都是真的退化”

- 也不是“全都只是你想多了”

为什么“官方偷偷 nerf 了我”也不够严谨

这种说法的问题在于，它把很多不同层的波动都压扁成一个解释。

更有用的判断方式是：

> 先问到底是哪一层变了，而不是先给动机下结论。

工程上怎么应对才有用

1. 保留可比样本

如果你真觉得质量波动了，尽量保留：

- 输入
- 上下文
- 输出
- 时间
- 模型版本

2. 做交叉验证

很实用的手段包括：

- 多上下文验证
- 多模型交叉验证
- 同任务重复几次

3. 不要只靠情绪判断

“今天感觉特别笨”可以是线索，但不能当结论。

对中文开发者最实用的启发

- 不要把每次失败都归因成“模型今天废了”
- 也不要把所有波动都归因成“是自己 prompt 不够好”
- 更像一个工程师那样拆问题：模型、路由、提示、上下文、任务难度

一句话总结

LLM 的日常波动既不是纯幻觉，也不等于“官方在偷偷 nerf 你”。
真正有用的做法，是把波动拆成可验证、可排查的工程问题。

第 4 部分：实现与样例

这一部分不是抽象概念，而是仓库里的真实落地样例。

读到这里，你应该开始能把“知识层”映射到“文件怎么组织、工作流怎么跑”。

最小工作流样例：Command → Agent → Skill

这篇为什么重要

整个上游仓库其实都在反复讲同一件事：

> Claude Code 不只是“会聊天”，而是可以被组织成工作流。

这篇就是那套工作流的最小可执行样例。

一张图讲清

```
用户
 ↓
command：负责入口和编排
 ↓
agent：负责自治执行
 ↓
skill：负责复用知识或生成产物
```

用 weather 样例解释

- `/weather-orchestrator`

- 问用户温度单位
- 调 `weather-agent`
- 再调 `weather-svg-creator`

- `weather-agent`

- 带着 `weather-fetcher` 这套预加载知识去取数

- `weather-svg-creator`

- 用已有上下文生成 SVG 和 markdown 输出

这个样例真正要教你的

不是怎么做天气卡片。

而是：

- 什么该放入口层
- 什么该放自治层
- 什么该放复用层

这三个层一旦分开，你后面再做：

- review 工作流
- spec 工作流
- 发布工作流
- 文档工作流

思路都会顺很多。

给中文读者的判断线

如果你在设计 Claude 工作流时脑子一团乱，可以先问自己这 3 个问题：

1 用户是从哪里点火的

2 哪一步应该丢给独立上下文自治完成

3 哪部分知识或步骤值得沉淀成可复用 skill

这篇样例就是这 3 个问题的标准示范答案。

Commands 实现样例

这篇看什么

看 command 在这个仓里到底承担什么角色。

代表性样例

`/weather-orchestrator`

它不是“万能命令”，而是一个非常标准的工作流入口：

1 问用户一个必要问题

2 调 agent 去取数

3 调 skill 去生成结果

4 汇总输出

为什么这个样例值得学

因为它让 command 做 command 该做的事：

- 组织流程
- 调度能力
- 兜住输出

而没有让 command 既像 agent 又像 skill，最后什么都沾一点。

一句话

如果你想知道“高质量 command 看起来应该像什么”，这个样例是很好的起点。

Skills 实现样例

这篇看什么

看这个仓库是怎么把“skill”做成两种不同角色的：

- 预加载给 agent 的 skill
- 在主流程里直接调用的 skill

两个代表

`weather-fetcher`

这类 skill 更像“预装知识”：

- 告诉 agent 去哪里取数
- 告诉 agent 从响应里拿什么
- 不负责写文件

`weather-svg-creator`

这类 skill 更像“动作单元”：

- 吃调用上下文里的输入
- 使用 supporting files
- 输出具体产物

这个仓库最值得抄的一点

`weather-svg-creator` 不把所有模板塞进 `SKILL.md`，而是拆到：

- `reference.md`
- `examples.md`

这非常适合中文团队，因为很多人第一次写 skill 最容易犯的错，就是把主文件写成一大坨说明书。

一句话

skill 的实现价值，不在“写多详细”，而在“主文件负责判断，参考文件负责细节”。

Subagents 实现样例

这篇看什么

这篇不是讲 subagent 的抽象概念，而是看这个仓库到底怎么把它落到文件里。

代表性样例

这个仓库用 `weather-agent` 做示范。

它的作用不是“展示一个复杂业务”，而是展示 subagent 最标准的一种职责：

- 不跟用户来回聊太多
- 自己根据预加载 skill 去完成一件自治任务
- 把结果交回主流程

这个样例为什么好

因为它边界非常清楚：

- 用户交互在 command

- 取数逻辑在 agent
- 生成结果文件在 skill

这让你能非常清晰地看见 subagent 在整个系统里的位置。

你真正该抄的不是天气本身

别抄：

- 迪拜
- 温度
- SVG 卡片

该抄的是：

- subagent 只负责一段自治职责
- skill 通过 `skills:` 预加载
- command 不直接越权替代 agent 干活

一句话

这个实现样例的价值，不在 weather，而在“职责切分的干净程度”。

Agent Teams 实现样例

这篇看什么

这篇不是在讲“多 agent 听起来多酷”，而是在讲：

> 当多个 Claude 会话像小团队一样协作时，仓库里应该长成什么样。

它和 subagent 的区别

最关键的一点：

- subagent 是单会话里的隔离上下文
- agent teams 是多个独立 Claude 会话协作

也就是说，agent teams 的粒度更大，协作感更强，维护成本也更高。

这个仓库里的样例

这里用的是一个 `time orchestration workflow`：

- command 负责编排
- agent 负责取时间
- skill 负责渲染 SVG

但更值得看的不是“时间卡片”本身，而是它怎么拆角色：

- Command Architect
- Agent Engineer

- Skill Designer

再通过共享任务清单去对齐数据契约。

这套实现最值得抄的地方

1. 角色拆分清楚

不是泛泛地说“你们几个并行干”，而是明确每个角色负责哪个文件、哪个接口。

2. 数据契约先对齐

这一点非常重要。
多 agent 协作最怕不是代码不会写，而是接口说不清楚。

3. 工作目录隔离

`agent-teams/.claude/` 的自包含设计很值得学，因为它不会污染仓库根部运行层。

一句话总结

agent teams 的实现价值不在于“并行”两个字，而在于：

> 角色、文件归属、数据契约和协作边界是否一开始就写清楚。

Scheduled Tasks 实现样例

这篇看什么

这篇展示的是：

> Claude Code 的 `/loop` 到底是怎么进入日常工作流的。

`/loop` 的实际价值

它不是“定时器玩具”，而是：

- 定时提醒
- 周期性检查
- 轻量监控
- 自动重复某类 prompt

在个人工作流里非常实用。

样例里发生了什么

这里用的是：

```
/loop 1m "tell current time"
```

它会：

- 把 `1m` 解析成每 1 分钟
- 创建对应的 cron 任务
- 在会话里周期性触发

这篇最值得记住的几个事实

- 最小粒度是 1 分钟
- 任务会自动过期
- 任务是 session scoped 的
- 随时可以取消

也就是说，它很适合“短期循环任务”，不适合拿来当永久调度系统。

对中文用户最有价值的启发

如果你经常做这些事：

- 每隔几分钟看部署状态
- 周期性跑某个简单检查
- 定时提醒自己看一个变化结果

那 `/loop` 很可能比你想象中有用得多。

一句话总结

`/loop` 的最佳位置不是“云端任务系统”，而是“会话内的轻量自动化助手”。

第 5 部分：工作流与协作

这一部分聚焦于更大粒度的流程设计。

从单条 prompt 到多阶段 workflow、多模型协作、多 agent 协作，都会在这里出现。

如果你已经不满足于让 Claude 写几个函数，而是想让它进入完整研发流程，这部分最重要。

Cross-model workflow

> Claude Code + Codex 的协作工作流
>
> 参考：
>
> - <https://github.com/shanraisshan/claude-code-best-practice>
> - <https://github.com/shanraisshan/codex-cli-best-practice>

这条工作流在解决什么问题

不是在回答“Claude 和 Codex 谁更强”，而是在回答：

> 如果我两个都在用，应该怎么分工才最顺手？

这条 workflow 的答案是：

1. Claude Code 做 plan
2. Codex 做 QA review

3 Claude Code 负责实现

4 Codex 做最终 verify

一眼看懂

```
PLAN      -> Claude Code
QA REVIEW -> Codex
IMPLEMENT -> Claude Code
VERIFY    -> Codex
```

每一步的职责

1. PLAN

在 Claude Code 里进入 plan mode。

目标：

- 把需求问清楚
- 形成 phased plan
- 明确测试门槛

2. QA REVIEW

让 Codex 读这份 plan，补：

- 漏掉的实现阶段
- 风险点
- 中间验证步骤

重点是“补强计划”，不是重写原计划。

3. IMPLEMENT

回到 Claude Code，按 phase 去实现。

4. VERIFY

最后再让 Codex 站在“审阅者”视角看结果有没有偏离计划。

对中文用户最重要的启发

- 多模型协作不是二选一
- 真正重要的是工位划分
- 让不同模型在不同阶段发力，往往比单模型硬顶更稳

参考图

RPI workflow

RPI = Research → Plan → Implement

这条 workflow 的核心思想很朴素：

不要在需求还没想清楚时就开始写。

它把事情拆成 3 个阶段，每个阶段都有明确产物和验证门槛。

为什么值得学

RPI 最有价值的地方不是目录结构，而是阶段边界：

- 先研究能不能做
- 再规划怎么做
- 最后实现

这样能大幅降低下面这类浪费：

- 写到一半推倒重来
- UI 和后端各写各的
- 实现完才发现验收标准没定义

流程图

目录结构

```
rpi/{feature-slug}/
├── REQUEST.md
├── research/RESEARCH.md
├── plan/PLAN.md
├── plan/pm.md
├── plan/ux.md
├── plan/eng.md
└── implement/IMPLEMENT.md
```

三个阶段分别做什么

Research

- 判断需求是否可行
- 判断是否值得做
- 先识别约束和风险

Plan

- 把产品、体验、工程拆开
- 形成可执行的阶段计划

Implement

- 按阶段落地
- 每阶段都要过验证

适合谁

- 需求不是一次性小修补
- 团队需要可追踪的中间产物

- 不想继续靠“边写边想”推进复杂功能

对中文团队最有价值的启发

- 先把 plan 变成工件，再让模型写代码
- 研究、规划、实现的职责不要混
- workflow 的价值不在“命令名”，而在“阶段门槛”

Agent Teams 提示词样例

为一个“迪拜时间 SVG 卡片”工作流创建一支 agent team。
这个工作流遵循 Command → Agent → Skill 架构：

- command 负责编排流程与用户交互
- agent 负责取实时数据
- skill 负责把结果渲染成可视化产物

重要要求：

- 所有文件都必须创建在 `agent-teams/.claude/` 内
- 不要写到仓库根部的 `.claude/`
- 这样这个示例才能保持自包含，并且能通过 `cd agent-teams && claude` 独立运行
- 不要直接复制现成 weather workflow，要从零搭建一条 time workflow

请分配这 3 个角色：

1 Command Architect

- 负责 `agent-teams/.claude/commands/time-orchestrator.md`
- 通过 Agent tool 调 `time-agent`
- 再通过 Skill tool 调 `time-svg-creator`
- frontmatter 使用 `model: haiku`
- 明确写出：顺序执行、正确工具使用、最终输出摘要
- 和其他角色对齐数据契约：`{time, timezone, formatted}`

1 Agent Engineer

- 负责：
- `agent-teams/.claude/agents/time-agent.md`
- `agent-teams/.claude/skills/time-fetcher/SKILL.md`
- 用 Bash 获取 Dubai 当前时间：
`TZ='Asia/Dubai' date '+%Y-%m-%d %H:%M:%S %Z'`
- 返回 `time`、`timezone`、`formatted`
- frontmatter 使用：
- `tools: Bash`
- `model: haiku`
- `color: blue`

- maxTurns: 3
- 通过 skills: 预加载 time-fetcher

1 Skill Designer

- 负责：
- agent-teams/.claude/skills/time-svg-creator/SKILL.md
- reference.md
- examples.md
- 接收 time、timezone、formatted
- 生成 agent-teams/output/dubai-time.svg
- 生成 agent-teams/output/output.md
- 只使用调用上下文提供的时间，不重新取数

团队协作要求

- 三个角色都要通过共享任务清单同步数据契约
- 契约固定为：{time, timezone, formatted}
- 可以并行启动，因为三部分基本独立，只需要在接口上对齐

第 6 部分：社区经验与导读

这一部分不是正式规范，而是高信号经验层。

适合在你已经理解基本概念之后，看高手和产品团队是怎么把 Claude Code 真正用顺的。

这里保留的是最值得中文开发者吸收的方法，而不是把原始长帖和长视频逐字搬过来。

Claude Code 团队的 10 条实战建议

这篇值不值得看

值得。
因为它不是某个人的“个人怪癖”，而更接近 Claude Code 团队内部的一组通用工作习惯。

如果你刚开始认真把 Claude Code 纳入日常开发，这篇是非常好的起点。

10 条建议，中文读者版

1 并行永远是第一生产力

多开几个 worktree，让多个 Claude 会话同时推进不同任务，这是团队反复强调的最高收益动作。

1 复杂任务先 plan，再写

不要一上来就让 Claude 开写。先把任务拆清楚，后面返工会少很多。

1 认真写 CLAUDE.md

这不是装饰文件，而是 Claude 长期理解项目的入口。

1 把你自己的 skill 做成仓库资产

不要每次重复打 prompt。高频动作值得沉淀成 skill 并进 git。

1 很多 bug，Claude 自己就能修

前提是你给了它足够上下文、足够验证方式，而不是一句“修一下”。

1 提示词能力要升级

不是拼字数，而是把问题、目标、上下文和约束讲清楚。

1 终端和环境很重要

Claude Code 不是一个纯 UI 产品，终端体验会直接影响效率。

1 该拆 subagent 时就拆

把复杂任务扔进独立上下文，主线程会干净很多。

1 Claude 不只是写代码，也能做数据和分析

如果你只拿它当代码生成器，价值还没吃满。

1 用 Claude 学会 Claude

让它帮你理解代码、总结模式、解释输出，本身就是一种学习加速。

对中文用户最有价值的启发

- 第一件该学的事不是“炫技巧”，而是并行、plan、memory 这三个基本动作。
- 中文用户最容易忽略的是：终端与工作树管理本身就是 Claude Code 工作流的一部分。
- 如果你只想先记一句话：

  把高频动作收敛成 skill，把复杂动作拆给 subagent，把重任务放进 plan mode。

适合谁看

- 刚从“随便试试”转向“认真高频使用” Claude Code 的人
- 想建立个人默认工作流的人
- 想知道团队级最佳实践从哪里起步的人

Boris 的 12 条 Claude Code 定制建议

这篇在讲什么

这篇不是教你“怎么用 Claude Code”，而是在讲：

> 你开始高频使用之后，哪些定制项最值得动手。

12 个定制方向

1 把终端环境配顺手

2 按任务调整 effort

3 装 plugins / MCP / skills

4 做自己的 custom agents

5 预先放行高频权限

6 打开 sandboxing

7 加 status line

8 定制 keybindings

9 挂 hooks

10 改 spinner verbs

11 用 output styles

12 把所有能改的都改成自己的风格

真正有长期价值的，只有前半段

如果你是中文开发者，我更推荐优先做这 6 件事：

- terminal
- effort
- MCP
- agents
- permissions
- sandbox

原因很简单：

- 这些会真实影响效率和安全边界
- 后面的 spinner、文案、外观更偏个性化，不是第一优先级

对中文用户的启发

- 定制不等于堆功能。
- 高质量定制应该先动“控制面”，再动“装饰层”。
- 如果你目前还是新手，别被花里胡哨的个性化内容带跑偏。

一句话总结

先把权限、模型、工具、agent 边界配对，再去玩个性化外观。

Boris 的 13 条 Claude Code 日常工作流建议

为什么这篇特别值得看

因为它最接近“一个真正高频用户每天怎么用 Claude Code”。

不是官方功能清单，也不是理想化教程，而是比较贴近日常工作的默认姿势。

13 条建议，压缩成中文版

1. 同时跑 5 个 Claude
2. 不够就继续并行到网页端
3. 复杂任务优先用更强模型和思考模式
4. 团队共享一份 `CLAUDE.md`
5. 在 PR 里用 @claude 反向更新规范
6. 大多数 session 都从 plan mode 开始
7. 把高频流程变成 slash command
8. 把常见流程自动化给 subagent
9. 用 PostToolUse hook 做自动格式化
10. 别乱用 `--dangerously-skip-permissions`
11. MCP 让 Claude 真正接上你的工具
12. 长任务交给后台 agent 验证
13. 永远给 Claude 一个验证自己工作的办法

对中文用户最重要的 3 条

如果你现在只想记 3 条，我建议是：

1. plan mode
2. shared `CLAUDE.md`
3. 给 Claude 一个验证路径

这 3 件事能直接把“ vibe coding ”拉向“ 工程化协作 ”。

适合谁看

- 想从“ 偶尔用 ”升级到“ 日常重度使用 ”的人
- 已经开始在团队里共享 Claude 工作方式的人
- 想知道高级用户默认姿势长什么样的人

Boris 的 15 个隐藏但高价值的 Claude Code 功能

这篇适合谁

适合已经把 Claude Code 用顺了的人。
如果你还在熟悉最基础的 agent / skill / command / settings，建议先别从这篇开始。

15 个点里，最值得先记住的

1. 移动端也能进 Claude Code
2. 会话可以在移动端 / 网页 / 桌面端 / 终端之间切换
3. `/loop` 和 `/schedule` 是非常强的自动化入口
4. hooks 可以让外部逻辑更确定地运行

5 Cowork Dispatch 代表了另一种协作方式

6 Chrome 扩展很适合前端工作

7 Claude Desktop 可以帮你自动起服务和测页面

8 `/branch` 可以把对话分叉出去

9 `/btw` 适合处理临时侧问题

10 git worktree 依然是并行开发核心

11 `/batch` 适合大批量改动

12 `--bare` 对 SDK 启动速度有帮助

13 `--add-dir` 可以给 Claude 更多目录访问范围

14 `--agent` 可以改变 Claude Code 的系统提示和工具边界

15 `/voice` 让语音输入进入工作流

对中文用户的现实建议

别一次学 15 个。
优先顺序建议是：

1 `git worktree`

2 `/loop`

3 `/batch`

4 `/btw`

5 `/branch`

这些最容易立刻产生“真的变快了”的感觉。

一句话总结

这篇的价值在于提醒你：

> Claude Code 的能力边界比你第一眼看到的大得多，但真正该先学什么，还是要按收益排序。

Code Review 与 Test Time Compute：Boris 的两条高信号建议

这篇其实在讲两件事

1. Code Review

Claude Code 的代码审查价值，不是“省一个 reviewer”，而是：

- 多 agent 视角并行审查
- 能补到人眼容易漏掉的问题
- 当代码产出速度大幅增加后，review 成为新的瓶颈

2. Test Time Compute

同一个模型在不同上下文窗口里做交叉验证，本身就是一种额外算力投入。

换句话说：

> 不是只有训练时算力重要，验证时怎么算、开几个上下文，也很重要。

对中文团队最有价值的启发

- 当 AI 产出代码越来越快，review 会越来越成为瓶颈
- “多上下文交叉验证”是一种非常实用的防幻觉方法
- 即使是同一个模型，放在不同上下文里也能互相纠错

适合谁看

- 开始把 Claude 写出来的代码送进 PR 流程的人
- 正在고민怎么提高代码审查质量的人
- 想理解“多个 context window 值不值得”的人

为什么要 Squash Merge，以及为什么 PR 要更小

这篇最核心的两个观点

1. 高速 AI 开发下，PR 越小越好

当产出速度上来后，大 PR 并不会显得“更高效”，反而会：

- 更难 review
- 更难回滚
- 更难定位问题

2. Squash merge 是更务实的默认选项

因为在高频 AI 辅助开发里，分支内会出现大量：

- fix lint
- retry
- 再试一次
- 小修小补

这些 commit 对主分支历史几乎没有信息价值。

对中文团队的启发

- AI 时代更要重视 PR 粒度
- “一个 PR = 一个可理解改动”比“保留所有中间提交”更重要
- 想让 review 真正跟上产出速度，小 PR 和 squash 几乎是标配

一句话总结

Claude 写得更快，不代表你可以把更多变化塞进一个 PR。
恰恰相反，越快越要把 PR 切小。

Thariq：Anthropic 内部是怎么用 Skills 的

> - Thariq 原文：<https://x.com/trq212/status/2033949937936085378>

这篇为什么重要

如果说 Boris 的内容更像“高频用户工作流”，那 Thariq 这篇更像：

> Anthropic 内部对 skill 这个能力边界的系统总结。

你最该记住的几个点

1. skill 不只是一个 markdown 文件

更准确地说，它是一个文件夹，可以带：

- 脚本
- 资产
- 示例
- 参考资料

2. skill description 是触发器，不是摘要

写 description 时，应该想的是：

- Claude 在什么场景下该触发它

而不是：

- 我想写一句多好看的简介

3. 好 skill 要做 progressive disclosure

不要把所有东西都塞进 `SKILL.md`。
主文件只保留高判断价值内容，重资料拆出去。

4. 要把 skill 当成可组合能力

不是一个 skill 解决所有问题，而是多个 skill 互相配合。

5. 要衡量 skill 是否真的被触发和被使用

做了 skill 不代表它真的在发挥作用。
触发率、误触发、没人用，都是要关心的。

对中文用户最实用的启发

- 别把 skill 做成“大而全说明书”
- 先解决触发条件，再解决内容完整度
- 能拆 supporting files 的，就不要都堆在主文件里

一句话总结

真正高级的 skill，不是写得最长，而是触发得准、边界清、能复用。

Boris 上 Lenny's Podcast：编码问题被解决之后，会发生什么？

这期讲什么

这期最有意思的地方，不是某个具体功能，而是 Boris
对“编码这件事如果被大幅自动化，工程师接下来会做什么”的判断。

你可以先记住的 5 个点

1 高强度用户已经不怎么手写代码了

2 并行 agent 会成为常态

3 产出速度上来后，瓶颈会转移到 review、协调和验证

4 工具设计不能只看今天的模型，要看几个月后的模型

5 Claude Code 更像工程操作系统，而不只是代码补全器

适合谁看

- 想理解 Claude Code 长期方向的人
- 想从“工具使用”切换到“工作方式升级”的人

不适合谁看

- 你现在只想知道怎么装、怎么登录、怎么写第一个 command

这种情况下先看本仓的 `tutorial/` 和 `best-practice/` 更划算。

Boris 上 The Pragmatic Engineer：Claude Code 是怎么做出来的

这期最值得听什么

这期更偏工程视角。
如果你关心的不是“怎么用”，而是“为什么这个产品会长成这样”，这期很值。

5 条高信号 takeaway

1 Claude Code 的设计不是围绕单个 prompt，而是围绕整个工程流程

2 并行、多 worktree、多 agent 是核心生产力假设

3 产品要对未来模型能力做提前设计

4 很多 today-best-practice 其实会随着模型进化继续重写

5 真正关键的是上下文、工具接入和验证，而不只是代码生成本身

适合谁看

- 已经熟悉 Claude Code 基础功能的人

- 想从产品和工程角度理解它的人
- 正在搭建团队工作流的人

Boris 上 Ryan Peterman：职业成长、产品判断与 Claude Code

这期更偏什么

这期不是纯工具介绍，更偏：

- 产品判断
- 职业成长
- 如何看待快速变化的模型能力

你可以重点关注

1 需求往往是潜伏的，不是用户一开始就会明确说出来

2 模型变化太快，很多“现在正确”的答案过几个月就会变

3 真正好的工具设计要给用户留可 hack、可扩展的空间

适合谁看

- 想理解 Boris 产品观的人
- 想知道 Claude Code 背后方法论的人

不适合谁看

- 你现在只想快速学会具体功能和命令

Boris 上 YC：为什么要为 6 个月后的模型设计

这期最有代表性的观点

> 不要只为今天的模型设计，要为 6 个月后的模型设计。

这几乎可以当成理解 Claude Code 演进方向的一把钥匙。

4 条高信号 takeaway

1 产品能力会随着模型快速重写

2 今天需要你显式 prompt 的事，几个月后可能不需要了

3 很多系统要设计成可被反复推倒重来

4 用户反馈和快速试错比早期架构洁癖更重要

为什么中文读者值得看

如果你在用 Claude Code 做自己的工作流、插件或团队规范，这期能帮你避免一个常见误区：

- 把今天的 workaround 误当成长期真理

Cat 与 Boris 上 Every：Claude Code 是如何被工程化出来的

这期为什么推荐排第一

如果你只想选一条长内容来理解 Claude Code 的产品底层逻辑，我最推荐这期。

它把下面几件事讲得很透：

- 为什么 CLI 是重要形态
- 为什么“可 hack”很关键
- 为什么 power user 行为能反过来定义产品方向
- 为什么 latent demand 对 AI 工具特别重要

5 条 takeaway

1. Claude Code 的本质是让模型获得工程师在终端里的真实行动能力
2. 高强度用户会天然把工具推到设计边界之外
3. 好的 AI 工具要允许被“滥用”出新工作流
4. 真正的产品信号来自重度用户，而不是平均用户
5. CLI 不是过渡形态，它可能本来就是很强的终局形态之一

适合谁看

- 做 AI 产品的人
- 正在给团队设计 Claude 工作流的人
- 想理解“为什么 Claude Code 长成这样”的人

第 7 部分：哪些值得直接抄，哪些不要直接抄

这一部分是为了避免你把中文仓库当成“万能模板”。

真正应该抄的是结构、方法和边界；不该盲抄的是过度个性化配置和仍在演进中的演示层。

值得直接抄的

- 学习路径设计
- `README` 入口导航
- 术语分层方式
- `command` / `agent` / `skill` 的职责切分
- 运行保护清单和本地校验脚本
- 工作流样例的拆法

先不要直接抄的

- 过度个性化的 spinner / status line 文案

- 仍在持续中文化的 presentation 细节
- 任何你自己都还没理解边界的运行时配置

一句话原则

先抄结构和方法，再抄具体配置。

第 8 部分：7 天学习路径与附录

最后这一部分把前面的大量内容收敛成行动路径。

如果你想把学习结果落回自己的项目里，这里最值得反复看。

7 天学习路径

Day 1

- 跑起来 Claude Code
- 完成第一次登录

Day 2

- 搞懂 `command / agent / skill`

Day 3

- 搞懂 `CLAUDE.md / memory / rules`

Day 4

- 搞懂 `settings / permissions / hooks / MCP`

Day 5

- 看懂最小工作流样例

Day 6

- 看 `tips` 和 `videos`

Day 7

- 在你自己的项目里，做出第一个真正可用的 command、skill 或 agent

附录：按目标挑内容

我只想快速学会

- 先看安装与登录
- 再看 Command / Agent / Skill 的分工
- 再看 Subagents 和 Skills 两章

我想搭自己的团队工作流

- 重点看 Settings、MCP、RPI workflow、Cross-model workflow

我想继续深挖

- 重点看深度 reports、实现样例，以及 upstream 原仓